KLASIFIKASI PENYAKIT CITRA DAUN KENTANG MENGGUNAKAN EKSTRAKSI FITUR HSV DAN GLCM DENGAN METODE K-NEAREST NEIGHBORS

Eva Ristiyanti (2318016)

LATAR BELAKANG DAN TUJUAN

Latar Belakang: Penyakit pada daun kentang dapat menurunkan kualitas dan kuantitas produksi pertanian secara signifikan. Identifikasi manual sering kali memerlukan waktu lama dan tenaga ahli, sehingga diperlukan sistem otomatisasi untuk mendeteksi penyakit sedari dini.

Tujuan: Membangun model machine learning yang mampu mengklasifikasikan kondisi daun kentang (sehat atau sakit) secara akurat berdasarkan fitur warna dan tekstur.

SUMBER DATASET

- **Sumber Dataset:** Data citra diperoleh dari Kaggle.
- **Isi Dataset:** Terdiri dari 100 citra daun kentang yang dibagi menjadi 3 kategori utama:
  - Bercak Awal (Early Blight).
  - Bercak Parah (Late Blight).
  - Sehat (Healthy).
- **Alasan Memilih Dataset:** Memiliki variasi visual yang jelas antara kondisi daun sehat dan yang terinfeksi, sehingga sangat representatif untuk studi kasus klasifikasi citra.

MENGAPA MENGGUNAKAN METODE KNN?

- **Sederhana dan Efektif:** KNN adalah algoritma yang mudah dipahami dan diimplementasikan tanpa memerlukan asumsi distribusi data yang rumit.
- **Berbasis Jarak:** Algoritma ini bekerja dengan mencari kemiripan berdasarkan perhitungan jarak (Euclidean) antara data uji dan data latih di dalam memori.
- **Fleksibel:** Sangat cocok digunakan untuk tugas klasifikasi citra yang telah melalui tahap ekstraksi fitur numerik seperti pada proyek ini.

IMPORT LIBRARY

```
from google.colab import drive
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from skimage.feature import graycomatrix, graycoprops
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
```

Blok kode ini berfungsi untuk memuat semua pustaka (library) yang diperlukan, mulai dari cv2 untuk pengolahan gambar, skimage untuk analisis tekstur, hingga sklearn untuk pemodelan KNN dan evaluasi akuras

LOAD DATASET

```python
dataset_path = '/content/drive/MyDrive/Colab Notebooks/dataset_penyakit_daun_kentang'
categories = ['bercak_awal', 'bercak_parah', 'sehat']

data_list = []

for category in categories:
    path = os.path.join(dataset_path, category)
    if os.path.exists(path):
        for image_name in os.listdir(path):
            full_path = os.path.join(path, image_name)
            data_list.append({'path': full_path, 'label': category})

df_path = pd.DataFrame(data_list)

print("Tampilan Tabel Alamat Gambar (Belum diekstrak):")
display(df_path.head()) # Menampilkan 5 data teratas
```

```
Tampilan Tabel Alamat Gambar (Belum diekstrak):
```

	path	label
0	/content/drive/MyDrive/Colab Notebooks/dataset...	bercak_awal
1	/content/drive/MyDrive/Colab Notebooks/dataset...	bercak_awal
2	/content/drive/MyDrive/Colab Notebooks/dataset...	bercak_awal
3	/content/drive/MyDrive/Colab Notebooks/dataset...	bercak_awal
4	/content/drive/MyDrive/Colab Notebooks/dataset...	bercak_awal

Sistem akan memindai folder kategori penyakit, mengumpulkan jalur (path) setiap gambar, dan menyusunnya ke dalam tabel data awal (DataFrame) beserta label kategorinya agar siap diproses pada tahap berikutnya

RUMUS EKSTRAKSI FITUR

```python
def extract_features(img_path):
    img = cv2.imread(img_path)
    if img is None: return None

    img = cv2.resize(img, (128, 128))

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    mean_r, mean_g, mean_b = np.mean(img_rgb, axis=(0, 1))

    img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mean_h, mean_s, mean_v = np.mean(img_hsv, axis=(0, 1))

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    glcm = graycomatrix(gray, distances=[1], angles=[0], levels=256, symmetric=True, normed=True)

    contrast = graycoprops(glcm, 'contrast')[0, 0]
    homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]
    energy = graycoprops(glcm, 'energy')[0, 0]
    correlation = graycoprops(glcm, 'correlation')[0, 0]

    return [mean_r, mean_g, mean_b, mean_h, mean_s, mean_v, contrast, homogeneity, energy, correlation]
```

Fungsi extract_features didefinisikan untuk mengubah gambar mentah menjadi 10 angka representatif, yang mencakup nilai rata-rata warna RGB, nilai rata-rata ruang warna HSV, serta empat fitur tekstur GLCM seperti kontras dan homogenitas.

EKSEKUSI EKSTRAKSI CITRA

```python
fitur_list = []

for index, row in df_path.iterrows():
    fitur = extract_features(row['path'])
    if fitur is not None:
        fitur.append(row['label'])
        fitur_list.append(fitur)

kolom = ['Mean_R', 'Mean_G', 'Mean_B', 'Mean_H', 'Mean_S', 'Mean_V',
         'Kontras', 'Homogenitas', 'Energi', 'Korelasi', 'Label']
df_final = pd.DataFrame(fitur_list, columns=kolom)

# Simpan CSV
df_final.to_csv('dataset_ekstraksi_eva.csv', index=False)

print("Tampilan Tabel Hasil Ekstraksi:")
display(df_final.head())
```

Tampilan Tabel Hasil Ekstraksi:

	Mean_R	Mean_G	Mean_B	Mean_H	Mean_S	Mean_V	Kontras	Homogenitas	Energi	Korelasi	Label
0	143.967773	150.132263	109.677856	64.888306	107.114319	155.706604	115.405512	0.398268	0.050542	0.977429	bercak_awal
1	90.412476	127.031982	111.067444	73.890808	116.296753	139.177063	193.161848	0.296355	0.031248	0.970907	bercak_awal
2	125.848206	140.063293	105.007385	62.456482	99.712524	143.170776	181.084338	0.264457	0.046248	0.975145	bercak_awal
3	101.939514	126.694519	118.453674	75.073608	82.241699	137.084473	159.143762	0.270225	0.026278	0.978150	bercak_awal
4	115.448547	131.962036	124.080322	71.886169	86.027466	143.325867	169.391609	0.349755	0.042799	0.977155	bercak_awal

Blok ini menjalankan fungsi ekstraksi ke seluruh gambar di dataset, lalu hasilnya disimpan ke dalam file CSV (dataset_ekstraksi_eva.csv) sehingga data citra kini resmi berubah menjadi tabel angka numerik.

PEMBAGIAN DATA DAN NORMALISASI

```python
X = df_final.drop('Label', axis=1)
y = df_final['Label']

# Split Data 70/30
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Normalisasi (Penting!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Data Latih = {X_train.shape[0]} baris, Data Uji = {X_test.shape[0]} baris.")
print("\nTampilan Tabel Data Latih SETELAH Normalisasi Z-Score:")

# Kita buat dataframe sementara khusus agar kamu bisa melihat hasil scaling-nya
df_scaled_preview = pd.DataFrame(X_train_scaled, columns=X.columns)
display(df_scaled_preview.head())
```

```
Data Latih = 70 baris, Data Uji = 30 baris.

Tampilan Tabel Data Latih SETELAH Normalisasi Z-Score:
```

	Mean_R	Mean_G	Mean_B	Mean_H	Mean_S	Mean_V	Kontras	Homogenitas	Energi	Korelasi
0	-0.418828	-0.390520	-1.038535	-0.017065	0.932406	-0.569264	1.579840	-1.230840	0.026955	-1.349167
1	0.031987	0.662348	1.343920	0.979033	-1.039570	0.601334	0.910606	-0.744174	-0.851775	-0.476132
2	-0.567302	-0.036864	-1.124698	-1.291209	0.903241	-0.402825	-0.322628	0.252802	-0.567312	0.914343
3	-0.202230	-0.041528	0.405766	0.275634	-0.638322	0.149772	-0.766502	0.046579	0.415837	0.083213
4	-1.630906	-0.710951	0.554547	1.171979	-0.078423	-0.205929	0.129898	-2.167178	-1.632563	-3.287558

Data dibagi menjadi 70% untuk belajar (latih) dan 30% untuk ujian (uji). Selanjutnya, sistem menerapkan StandardScaler untuk menormalkan skala angka fitur agar fitur warna dan tekstur memiliki rentang nilai yang seragam.

PELATIHAN MODEL KNN

```python
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)

akurasi = accuracy_score(y_test, y_pred)
print(f"Tingkat Akurasi Model KNN: {akurasi * 100:.2f}%\n")
print("Laporan Klasifikasi Detail:")
print(classification_report(y_test, y_pred))
```

```
Tingkat Akurasi Model KNN: 63.33%

Laporan Klasifikasi Detail:
              precision    recall  f1-score   support

 bercak_awal       0.67      0.91      0.77        11
bercak_parah       0.44      0.50      0.47         8
       sehat       0.83      0.45      0.59        11

    accuracy                           0.63        30
   macro avg       0.65      0.62      0.61        30
weighted avg       0.67      0.63      0.62        30
```

Model KNN dengan parameter 5 tetangga terdekat (n_neighbors=5) dilatih menggunakan data latih yang sudah dinormalisasi. Setelah itu, model langsung diuji untuk memprediksi data baru dan menghitung skor akurasinya

PENERAPAN PERHITUNGAN MATEMATIS

A. Ekstraksi Fitur (Mengubah Gambar ke Angka)

1. Fitur Warna (Mean Hue)

Komputer menjumlahkan semua nilai warna piksel pada ruang warna *Hue* dan mencari rata-ratanya.

Rumus:

$$MeanH = \frac{1}{n}\sum_{i=1}^{n} H_i$$

- **Contoh:** Jika kita memiliki 4 piksel sampel dengan nilai Hue: 60, 62, 58, 60.
- **Perhitungan:** $\frac{60+62+58+60}{4} = \mathbf{60}$.

2. Fitur Tekstur (Contrast GLCM)

Menghitung tingkat kekasaran atau perbedaan intensitas antar piksel yang bertetangga menggunakan matriks GLCM.

Rumus:

$$Contrast = \sum_{i,j=0}^{N-1} P_{i,j}(i-j)^2$$

- **Logika Sederhana:** Pada **Daun Sehat**, perbedaan antar piksel sangat kecil (mulus), sehingga nilai kontrasnya rendah.
    - Pada **Daun Sakit (Bercak)**, perbedaan intensitas piksel sangat tajam (kasar), sehingga nilai kontrasnya tinggi.

PENERAPAN PERHITUNGAN MATEMATIS

B. Normalisasi Z-Score (Penyetaraan Skala)

Karena nilai fitur warna (puluhan) dan tekstur (ratusan/ribuan) memiliki rentang yang berbeda, kita harus menyetarakan skalanya menggunakan **StandardScaler** agar algoritma KNN dapat memberikan bobot yang adil pada setiap fitur.

Rumus:

$$Z = \frac{x - \mu}{\sigma}$$

- **Keterangan:** x (nilai asli), μ (rata-rata), σ (standar deviasi).
- **Contoh:** Misal nilai Kontras daun adalah **2500**, rata-rata dataset adalah **1500**, dan standar deviasi **500**.
- **Perhitungan:** $Z = \frac{2500-1500}{500} = \frac{1000}{500} = \mathbf{2.0}$.

PENERAPAN PERHITUNGAN MATEMATIS

C. Perhitungan Jarak Euclidean (Logika KNN)

Untuk menentukan kategori daun baru, komputer menghitung jarak terdekat antara **Daun Baru** dengan **Data Latih** di dalam memori.

Rumus Jarak (Euclidean Distance):

$$d(A, B) = \sqrt{(H_1 - H_2)^2 + (C_1 - C_2)^2}$$

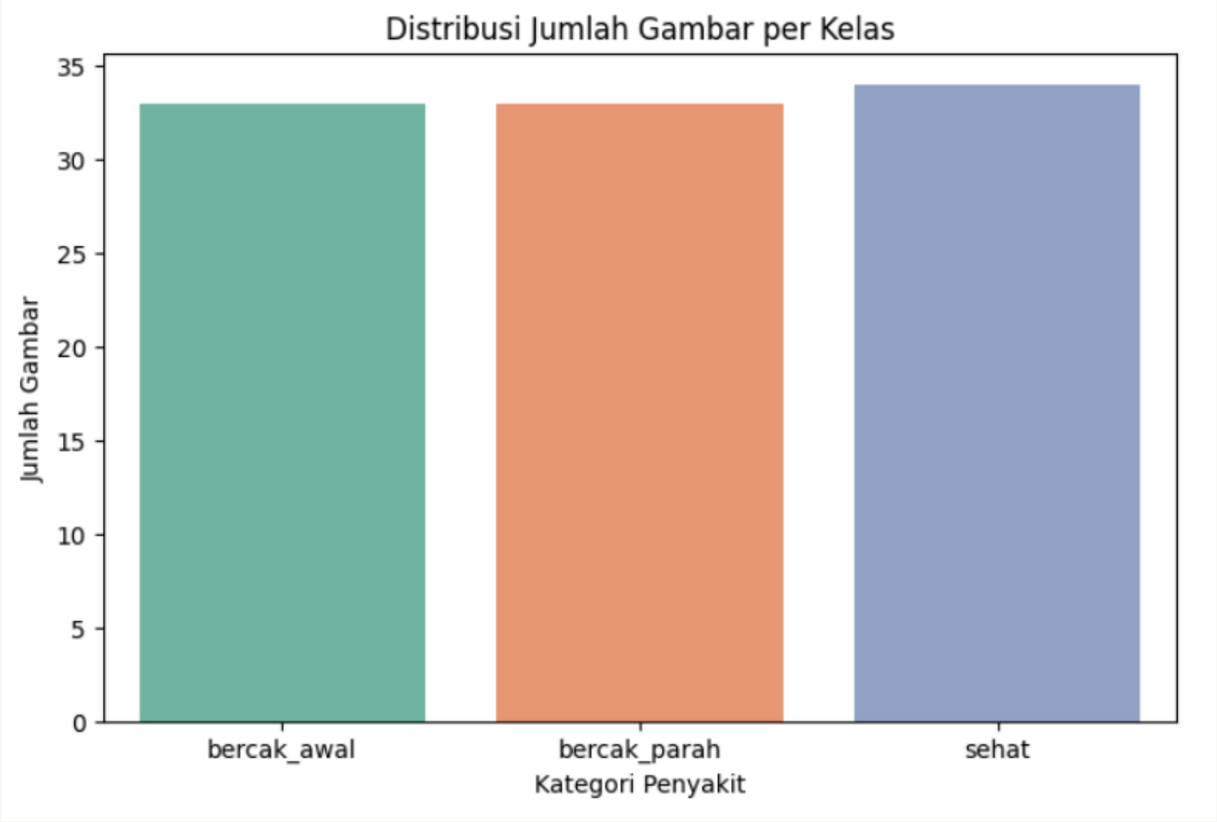
Simulasi Perhitungan Manual:

Kita memiliki **Daun Baru** dengan nilai normalisasi: **Hue = 0.5** dan **Kontras = 1.2**. Kita bandingkan dengan data di memori komputer:

Data Latih	Label	Nilai Hue (Z)	Nilai Kontras (Z)	Perhitungan Jarak Euclidean (d)
Data A	Sehat	0.6	0.2	$\sqrt{(0.5-0.6)^2+(1.2-0.2)^2} = \mathbf{1.00}$
Data B	Sakit	0.4	1.1	$\sqrt{(0.5-0.4)^2+(1.2-1.1)^2} = \mathbf{0.14}$

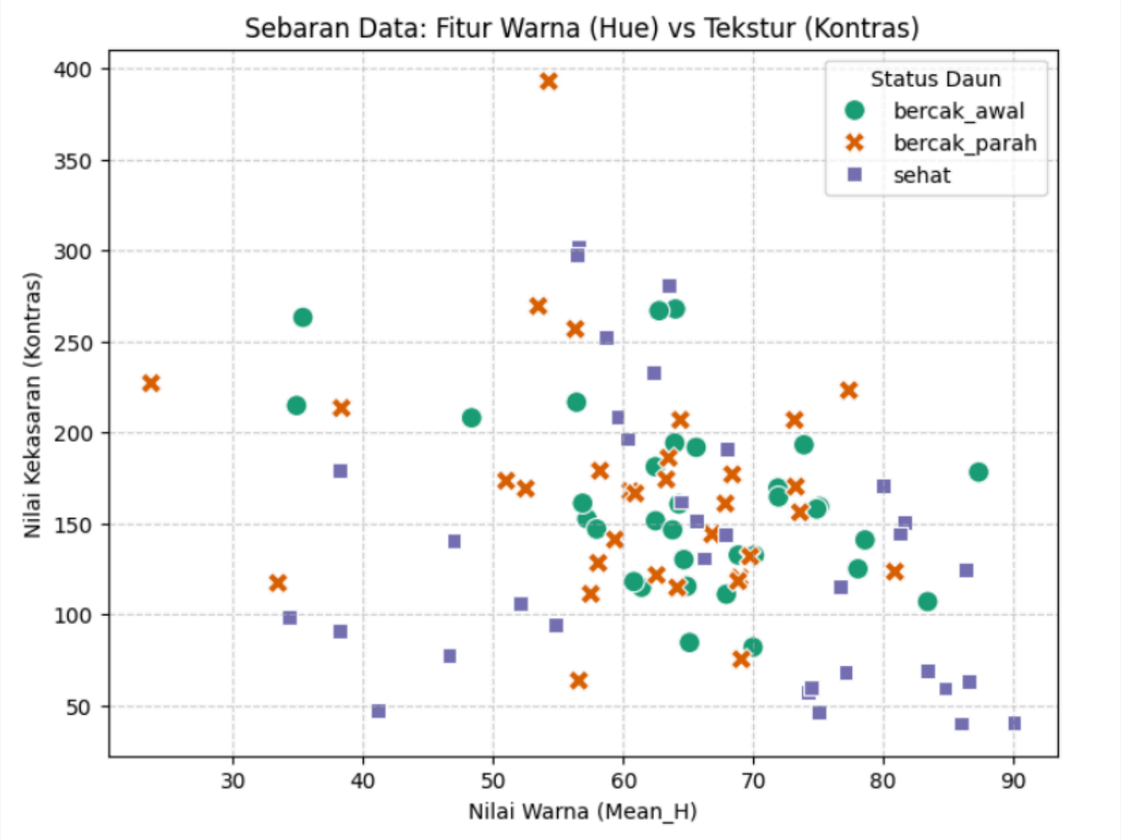
Kesimpulan: Karena jarak ke **Data B (0.14)** jauh lebih kecil dibandingkan ke Data A (1.00), maka algoritma **KNN** akan memprediksi bahwa Daun Baru tersebut berstatus **SAKIT**.

EVALUASI VISUAL DISTRIBUSI DATA



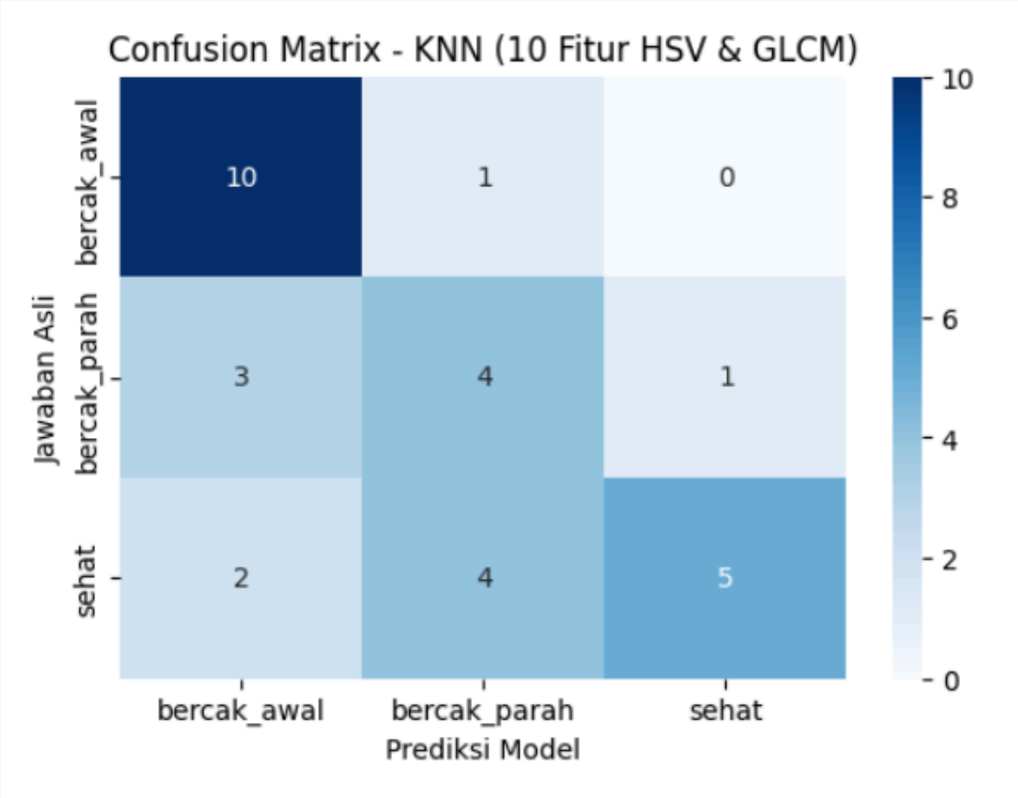
Grafik ini menampilkan distribusi seimbang dari 100 dataset citra daun kentang yang terbagi ke dalam tiga kategori: bercak awal (bercak_awal), bercak parah (bercak_parah), dan sehat (sehat). Setiap kelas memiliki frekuensi hampir seragam, yaitu sekitar 33 hingga 34 gambar . Kondisi dataset yang seimbang ini sangat krusial untuk mencegah bias pada algoritma K-Nearest Neighbors (KNN) agar model dapat mempelajari fitur setiap kelas secara adil selama tahap pelatihan.

EVALUASI VISUAL SCATER PLOT



Grafik scatter plot ini memvisualisasikan sebaran data daun kentang berdasarkan Nilai Warna (Mean_H) dan Nilai Kekasaran (Kontras) . Visualisasi ini bertujuan untuk mengamati pemisahan atau pengelompokan karakteristik visual antara kategori bercak awal, bercak parah, dan sehat . Pola distribusi titik-titik tersebut menjadi dasar bagi algoritma KNN untuk menghitung jarak Euclidean dalam menentukan klasifikasi tiap citra .

EVALUASI VISUAL SCATER PLOT



Gambar Confusion Matrix ini merangkum efektivitas model KNN dalam mengklasifikasikan 30 data uji citra daun kentang. Berdasarkan matriks tersebut, model memiliki performa paling tinggi pada kategori bercak_awal dengan tingkat keberhasilan 10 dari 11 citra. Sementara itu, model berhasil memprediksi 4 data bercak_parah dan 5 data sehat secara akurat, meskipun masih terdapat beberapa kekeliruan prediksi antar kategori akibat kemiripan fitur visual. Secara keseluruhan, penggunaan 10 fitur HSV dan GLCM ini memberikan kontribusi pada tercapainya nilai akurasi sebesar 63.33%.

TERIMA KASIH